

Основные определения

Технология программирования в Интернет

2 курс, ПИ • 2026

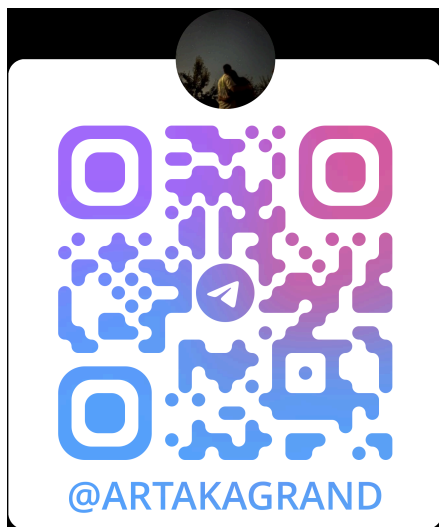
Составил: Мышковец Артём 2-10

tg: @artakagrand

t.me/artakagrand (Нажми)

(Если не работает скопировать и вставить в браузер)

или



Предисловие

Этот файл не 100% верный сборник определений, а просто приложение к ответам на вопросы. Тут собраны по возможности все определения, которые звучали в лекциях и упоминались в вопросах к экзамену.

Файл сделан в формате шпаргалки для проверки или приобретения основных знаний. Если вам что-то не понятно, то смотрите большой файл с ответами на вопросы, возможно там станет понятнее.

Кто хочет сказать спасибо или поддержать:

4246410033771210

1. Интернет и HTTP

Интернет — всемирная компьютерная сеть, построенная на основе стека протоколов TCP/IP. Включает в себя сеть TCP/IP, службы Интернет, организации, обслуживающие интернет и документацию (RFC, STD).

Протокол — правила обмена данными.

От автора: протокол это такой набор четко определенных правил использующихся там, где используется этот протокол. Таким образом протокол декларирует что будет делать и как себя вести объект протокола. Частный случай это протокол

HTTP, т.е. в нем описаны все правила передачи информации, команд, формат сообщений, коды состояния, поведение клиентов и серверов и тд.

Служба (сервис) Интернет — сервер (программа, слушающая порт) + протокол. Слушает порт 1–1023. Порты с 1 по 1023 официально называются **«хорошо известные порты» (Well-known ports)**. Примеры служб: E-mail, FTP, DNS, DHCP.

Сокет — объект операционной системы состоящий из двух компонентов: IP-адрес и порт (число от 1 до 2^{16}).

Интернет-ресурс — сущность в сети Интернет, имеющая свой адрес в сети.

От автора: Web ресурс приложения: сущность, расположенная на стороне сервера и имеющая URL/URI, к которой можно сделать http-запрос и получить http-ответ. Одно web-приложение представлено одним или более ресурсов.

Интернет-сервер — программа, которая прослушивает сетевой порт и обрабатывает запросы.

HTTP — клиент-серверный протокол обмена данными посредством сообщений.

Типы сообщений HTTP — только два: request (от клиента) и response (от сервера). 1 request = 1 response. Без request не будет response.

URI(Uniform Resource Identifier) — унифицированный идентификатор ресурса (стандарт STD 66). URI является либо указателем ресурса URL(Uniform Resource Locator) либо именем ресурса URN(Uniform Resource Name), либо одновременно обоими.

Пользовательский заголовок — заголовок, придуманный программистом; имя начинается с «X-».

Статусы ответа — 1xx — инфо; 2xx — успех; 3xx — переадресация; 4xx — ошибка клиента; 5xx — ошибка сервера.

2. Web-приложение

Web-приложение — клиент-серверное приложение, у которого клиент отправляет запросы по протоколу HTTP, а сервер отправляет ответы по протоколу HTTP.

Клиент — инициатор связи, делает запрос (частный случай — браузер).

Сервер — обрабатывает запросы и отдаёт обратно клиенту ответы.

Web-программирование — разработка клиент-серверных приложений, компоненты которых взаимодействуют по HTTP; частный случай программирования в Интернет.

Маршрутизация — механизм выбора обработчика HTTP-запроса (чаще всего по методу и URI).

DOM — модель представления браузером HTML-страницы + API для доступа к ней (через объект document).

3. ASP.NET Core: общее

ASP.NET Core — программная платформа (кроссплатформенный фреймворк) для разработки веб-приложений на .NET.

Платформа — набор, состоящий из: язык программирования + сборщики и компиляторы для ЯП + библиотеки + описание API + инструменты разработки.

Кроссплатформенность — возможность запускать программу на разных ОС. Бывает на уровне исходного кода и на уровне байт-кода (**у ASP.NET Core — байт-код**).

Host (хост) — программа-процесс ОС, образующая исполняемый файл; внутри неё логика приложения. Программа которая может создавать процесс.

Kestrel — кроссплатформенный HTTP-сервер в составе ASP.NET Core; создаёт объект HttpContext.

HttpContext — объект, создаваемый HTTP-сервером; содержит всю информацию о соединении, запросе и будущем ответе (Request, Response).

wwwroot — директория для статических файлов; отсюда они по умолчанию отдаются клиенту.

NuGet — программный менеджер пакетов (пакет = одна или несколько библиотек).

Сервис — класс (объект класса) с заданным жизненным циклом, создаётся автоматически; содержит бизнес-логику.

4. Middleware и обработка ошибок

Middleware — двунаправленный конвейер обработки запросов и ответов; реализация паттерна «цепочка ответственности».

Принцип middleware — вложенные вызовы: до next — обработка запроса, после next — обработка ответа.

Обработчик ошибок — middleware-элемент в начале конвейера; до него «докатывается» необработанное исключение по стеку вызовов, если не обработается до него.

Распространение исключения — если нет обработчика, исключение поднимается вверх по стеку вызовов, ища обработчик, а в случае если такой не найдется, то клиенту вернется 500-я ошибка и ошибка запишется в логи сервера.

5. Статические файлы

Статический файл — файл на стороне сервера, который без изменения может быть переслан клиенту (HTML, CSS, JS, картинки, видео, музыка и тд).

UseStaticFiles — middleware-элемент, обеспечивающий выдачу статических файлов.

Получение статики — теги `<link>`, `<script>`, `<img>` приводят к GET-запросу за статическим ресурсом.

6. Minimal API

Minimal API — приложение из простейших конечных точек с простейшей маршрутизацией (по методу и URI).

Конечная точка (endpoint) — HTTP-обработчик, возвращающий ответ (MapGet, MapPost, ...).

Fallback-обработчик — запасная точка, срабатывает, если ни одна другая точка не подошла.

MapGroup — задаёт общий префикс для группы конечных точек.

Content-Type — заголовок, указывающий формат тела сообщения.

Accept — заголовок, которым клиент сообщает, в каком формате хочет/может принять ответ.

Фильтр конечной точки — обработчик вокруг точки, работает до и после неё; аргументы — через GetArgument, переход — через next.

Шаблон маршрута — символическое выражение, задающее множество обрабатываемых URI.

Констрейнт (constraint) — ограничение на параметр маршрута (тип, диапазон, длина): `{id:int}`, `{key:guid}`, `{age:min(18)}`.

7. Переадресация

Переадресация (Redirect) — сервер не может/не хочет обработать запрос, но знает где можно. Он отвечает кодом 3xx и заголовком Location с новым URI. Клиент обязательно делает новый запрос по этому URI.

Коды 301, 302 — клиент делает новый запрос методом GET.

Коды 307, 308 — клиент делает тот же запрос с тем же методом (сохраняет тело и параметры), по новому адресу.

Шаблон ответа:

Если сервер отвечает кодом 3xx и заголовком Location, клиент **обязан** сделать новый запрос по указанному URI. При 301/302 этот новый запрос **всегда будет методом**

GET. При 307/308 клиент **обязан сохранить исходный метод** (например, POST), а также полностью сохранить тело запроса и все параметры.

Подробнее см. файл с ответами на вопросы

8. Привязка модели

Привязка модели (Model binding) — механизм, ставящий в соответствие параметры запроса параметрам вызываемого метода (по именам). Иначе — преобразование запроса в POCO-объект.

POCO — Plain Old CLR Object — обычный объект, может содержать логику.

DTO(Data Transfer Object) — класс без логики, только данные, для пересылки между частями приложения.

Атрибуты FromXXX — подсказывают, откуда брать параметр: `[FromRoute]` , `[FromQuery]` , `[FromHeader]` , `[FromBody]` , `[FromServices]` .

TryParse — метод в собственном типе; проверяет строку из URI — можно ли её распарсить (выбрать/обойти точку).

Хорошо известные типы — типы, автоматически распознаваемые как параметры: `HttpContext`, `HttpRequest`, `HttpResponse` и др.

Пользовательская привязка — свой механизм через статический метод `BindAsync`.

[AsParameters] — привязывает все параметры к свойствам класса/структуры (внутри можно использовать `FromXXX`).

Валидация — проверка пришедших данных на корректность.

Встроенная валидация — атрибуты в модели: `Required`, `Min`, `Max`, `MaxLength`, `e-mail`, регулярное выражение (`MinimalApis.Extensions`).

9. Dependency Injection

Dependency Injection (DI) — механизм получения объекта сервиса в автоматическом режиме (через параметры) с заданным жизненным циклом. Частный случай IoC.

Service Locator — сами «вытаскиваем» сервис: `GetService<T>` , `GetRequiredService<T>` (напр. `LinkGenerator`).

Transient — создаётся при каждом обращении (всегда новый).

Scoped — в рамках одного запроса(вызовов функции обработчика) — как `Singleton`, в разных запросах(вызовах функции обработчика) — как `Transient`. Предполагает что после выхода жизни он очищается.

Singleton — создаётся один раз, всегда один и тот же объект.

[FromServices] — подсказывает взять параметр из сервисов (DI).

10. Razor Page

Razor Page — компонент ASP.NET Core, обрабатывающий cshtml-файлы; многостраничное приложение с рендерингом на сервере.

MVVM — паттерн: View (cshtml-разметка) + Model (PageModel с данными и обработчиками).

Жизненный цикл cshtml — при первом обращении компилируется в C#-объект (генерирует разметку); далее используется готовый объект (runtime).

@page — директива: это страница; здесь же можно указать маршрут.

@model — задаёт тип модели — тип параметра представления.

@inject — внедрение зависимости (DI) в представление.

@using — сокращение имён namespace (как using в коде).

@functions — объявление функций в рамках представления.

Макет (Layout) — cshtml-файл — общий каркас, применяемый к макетируемой странице (переменная Layout).

@RenderBody() — место в макете для основного содержимого страницы.

@RenderSection(имя, false) — место для именованной секции (false — необязательная).

@section имя {...} — в странице задаёт содержимое для соответствующего RenderSection.

Форма — HTML-тег `<form>`; параметры отправляются в теле POST-запроса (имя — из name, значение — из value).

Upload — пересылка файлов с клиента на сервер; формат multipart/form-data; на сервере — IFormFile.

Tag helper — инструмент, замаскированный под HTML-тег; узнаётся по особым атрибутам (asp-for, asp-append-version).

Partial (частичное представление) — отдельное представление, вставляемое в другое для сокращения разметки.

11. MVC

MVC — архитектурный паттерн: Model (данные) + View (отображение) + Controller (обработка запросов). Маршрутизатор — выбор контроллера и действия.

Назначение MVC — сделать модель и контроллер максимально независимыми.

Контроллер — класс обработки запросов; имя = имя + суффикс Controller.

Жизненный цикл контроллера — создаётся на каждый запрос, умирает после каждого запроса.

Action (действие) — публичный метод контроллера, вызываемый по URI.

Таблица маршрутизации — набор шаблонов маршрутов; проверяются сверху вниз; по умолчанию Home/Index.

Шаблон маршрута MVC — `{controller}/{action}/{id?}` — 2-х или 3-х компонентный URI.

Фильтр (Action-фильтр) — класс, реализующий интерфейс; работает до и/или после действия; привязан к контроллеру/действию.

DI в контроллере — два способа: через конструктор или через `HttpContext.Services` (Service Locator).

ViewData — коллекция «ключ–значение» на один запрос (доступ через ключ).

ViewBag — та же коллекция, что ViewData, но доступ через свойства.

Представление (View) — cshtml-файл, вызывается из контроллера, генерирует разметку ответа.

Html-helper — метод, генерирующий кусочек HTML; бывает встроенный (`Html.ActionLink`) и пользовательский.

12. Web API / REST

REST — архитектурный стиль взаимодействия компонентов. Представление = URI, управление = глаголы HTTP (GET, POST, PUT, DELETE). Автор Roy Fielding, 2000.

Web API — приложение (по сути MVC), возвращающее JSON или статус, без View; способ реализации REST.

RESTful — веб-служба, поддерживающая REST-интерфейс в полном объёме.

Соответствие методов — GET — select, POST — insert, PUT — update, DELETE — delete.

Типы ресурсов — коллекция (`/api/users`) и элемент коллекции (`/api/users/288`).

HATEOAS — гипермедиа как управление состоянием приложения.

Идентификация — заявление пользователя о себе.

Аутентификация — проверка подлинности идентификации пользователя.
Проверяется заявил ли пользователь о себе как о том, кем он является.

Авторизация — проверка прав аутентифицированного пользователя.